

Orquestación y Despliegue de Agentes





Agenda de la sesión

BLOQUE 01

Contexto: del notebook al mundo real

10 min

BLOQUE 02

Orquestación de agentes

20 min

BLOQUE 03

Despliegue en producción

20 min

BLOQUE 04

Proyecto final: instrucciones y rúbrica

10 min



Del notebook al mundo real

Notebook / Demo

- × Funciona en local
- × API key en texto plano
- × Una conversación a la vez
- × Reinicio manual si falla
- × Sin logs ni monitorización



Producción

- ✓ Accesible via API / web
- ✓ Secretos gestionados
- ✓ Concurrencia y escalado
- ✓ Reintentos automáticos
- ✓ Logs, métricas y trazas

Los 3 grandes retos del despliegue



Latencia y Coste



Llamadas al LLM son caras y lentas

Caching de embeddings y respuestas frecuentes

Elegir el modelo correcto para cada tarea

Fiabilidad



El LLM puede fallar o hacer timeout

Reintentos con backoff exponencial

Fallback a respuestas predefinidas

Seguridad



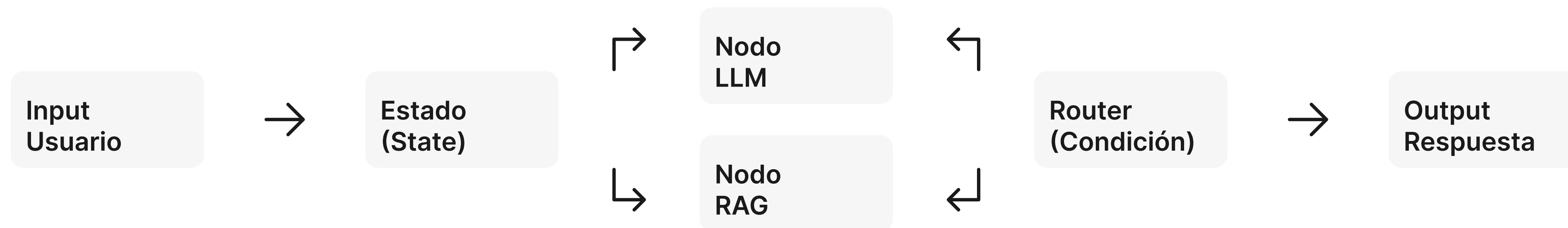
Nunca exponer API keys en el código

Validar y sanitizar inputs del usuario

Rate limiting para evitar abuso



Repaso: arquitectura de un agente LangGraph



Estado: diccionario compartido entre nodos

Edges condicionales: deciden el siguiente nodo

Grafo: define el flujo de control

Checkpointing: guarda estado entre llamadas



Patrones de orquestación: multi-agente

Agente Único

Un grafo con múltiples nodos y herramientas. Simple y fácil de depurar. Ideal para el proyecto.

✓ Recomendado para el proyecto

Supervisor

Un agente orquestador delega tareas a sub-agentes especializados. Bueno para tareas complejas.

→ Proyectos avanzados

Pipeline

Agentes en secuencia: salida de uno es entrada del siguiente. Ideal para workflows predecibles.

→ Procesos ETL / batch



Gestión del estado en producción



El problema

Los LLMs no tienen memoria entre llamadas

Cada request llega sin contexto previo

Si el servidor cae, la conversación se pierde

Múltiples usuarios comparten la misma instancia



La solución: checkpointing

MemorySaver

En RAM, para desarrollo y demos

SqliteSaver

Persistencia local en SQLite

PostgresSaver

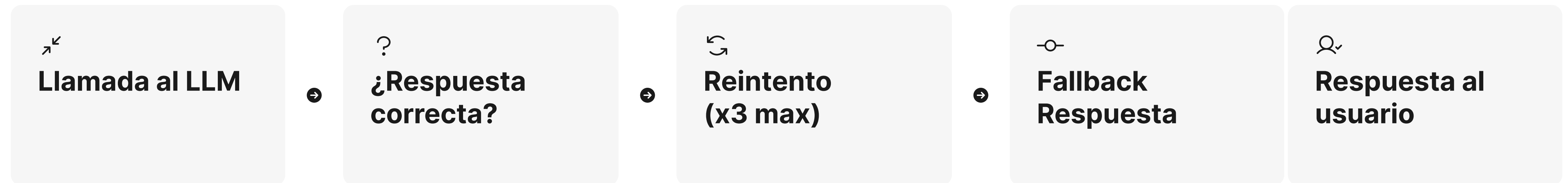
Producción multiusuario

thread_id

Identifica cada conversación



Manejo de errores y reintentos



Patrón de reintento con tenacity

```
from tenacity import retry, stop_after_attempt, wait_exponential
```

```
@retry(stop=stop_after_attempt(3),  
      wait=wait_exponential(multiplier=1, min=2, max=10))
```

```
def call_gemini(prompt: str) → str:  
    return llm.invoke(prompt)
```



Observabilidad: trazas y métricas con LangSmith

Plataforma de observabilidad para aplicaciones LangChain/LangGraph. Permite ver en tiempo real cómo el agente toma decisiones, qué tokens consume y dónde falla.

Latencia

Tiempo por nodo y por run completo

Tokens

Coste real de cada llamada al LLM

Errores

Tasa de fallos y mensajes de error

Feedback

Thumbs up/down de los usuarios

Setup en 3 líneas de código

```
import os
os.environ["LANGCHAIN_TRACING_V2"] = "true"
os.environ["LANGCHAIN_API_KEY"] = "ls__..." # tu API key de LangSmith
```

A partir de aquí, cada invocación del agente queda trazada automáticamente



Opciones de despliegue

Local / Notebook	Streamlit Cloud	FastAPI + Cloud Run
Ventajas Sin coste Fácil de depurar Ideal para el proyecto	Ventajas Deploy en 1 clic desde GitHub URL pública gratuita Ideal para el bonus del proyecto	Ventajas API REST estándar Escalado automático en GCP Producción real
Limitaciones No accesible externamente Sin escalado	Limitaciones Recursos limitados (free tier) Cold starts lentos	Limitaciones Más configuración inicial Coste según uso



Empaquetar el agente: FastAPI

FastAPI convierte tu agente en una API REST que cualquier cliente puede consumir — incluyendo Streamlit.

```
from fastapi import FastAPI
from pydantic import BaseModel
from agent import run_agent # tu grafo LangGraph

app = FastAPI()

class Query(BaseModel):
    question: str
    thread_id: str = 'default'

@app.post('/chat')
async def chat(query: Query):
    response = run_agent(query.question, query.thread_id)
    return {'answer': response}
```

```
▶ uvicorn main:app --host 0.0.0.0 --port 8080
```



Secretos, escalado y control de costes



Gestión de API Keys

Nunca hardcodear keys en el código

Desarrollo: archivo .env + python-dotenv

Producción: Secret Manager (GCP/AWS)

Rotar keys si se exponen accidentalmente



Escalado básico

Rate limiting: máx. N requests/min por usuario

Async endpoints en FastAPI para concurrencia

Semaphore para limitar llamadas paralelas al LLM

Queue (Celery / Cloud Tasks) para batches



Control de costes

Usar gemini-1.5-flash para consultas simples

Cachear respuestas frecuentes (Redis / dict)

Limitar max_tokens en cada llamada

Monitorizar gasto en Google AI Studio



Inputs del usuario

Limitar longitud máxima del mensaje

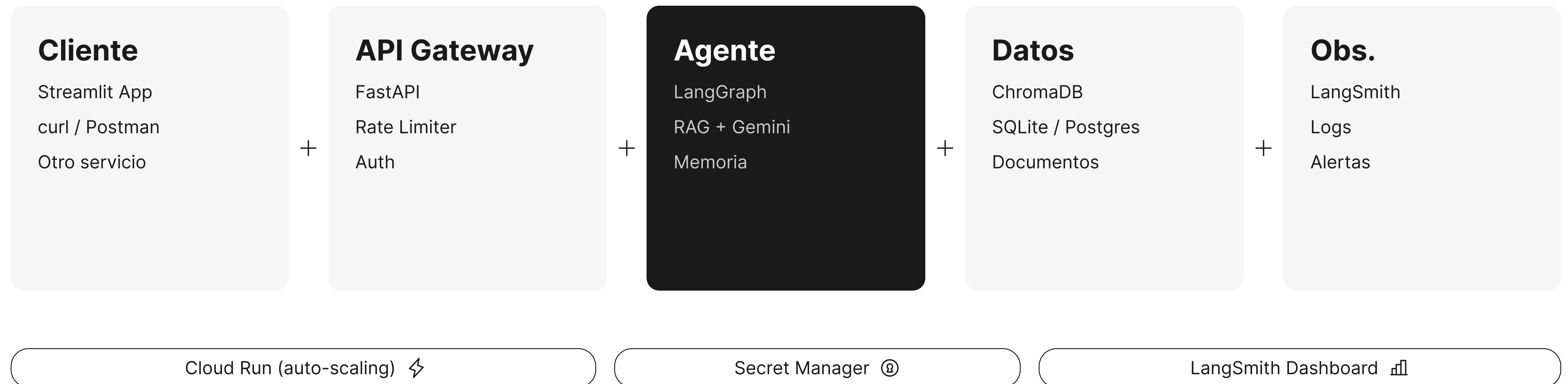
Sanitizar antes de insertar en el prompt

Guardar logs de conversaciones

Bloquear intentos de prompt injection



Arquitectura end-to-end: agente en producción





Proyecto Final — IA Generativa: Asistente Experto con Gemini, RAG y Agentes



Construye un agente LangGraph con base de conocimiento vectorial sobre un tema que elijas.

Stack tecnológico:

Google Gemini 1.5

ChromaDB

LangGraph

Jupyter Notebook

Streamlit(Opcional)

MVP — Lo mínimo que debes entregar:

- Base de conocimiento en ChromaDB (mín. 3 docs / ~20 páginas) con Gemini Embeddings
- Agente LangGraph con RAG + Gemini como LLM + memoria de conversación
- System prompt documentado y justificado en el README
- Notebook funcional con celda de chat y 5 ejemplos de conversación



Rúbrica de evaluación

Criterio	Excelente	Correcto	Mejorable
Base de conocimiento (ChromaDB + Embeddings)	Indexación correcta, mín. 3 docs / ~20 págs.	Funciona pero docs escasos	Errores o sin contenido
RAG + Gemini	Recuperación precisa, respuestas coherentes	Funciona pero poco preciso	No integrado o roto
Ingeniería de Prompts	System prompt justificado en el README	Prompt básico, sin justific.	Sin prompt personalizado
Agente + Memoria (LangGraph)	Memoria funcional, coherencia entre turnos	Memoria básica o parcial	Sin memoria
Código y Documentación	Código limpio, README con instrucciones claras	Funcional, docs mínimas	Difícil de leer / sin docs
Presentación + Demo (5-10 min)	Demo fluida, decisiones técnicas bien explicadas	Demo funciona, explicación básica	Errores o sin presentar
BONUS: Streamlit	Funcional y desplegada	Solo local	—



Entregables y calendario

Hoy
Jueves S2
Kick-off del proyecto

Martes S3
Tutoría resolución de dudas

Miércoles S3
Entrega del notebook y
README

Jueves S3
Presentaciones
(5-10 min + demo)

Entregables obligatorios:

-  Notebook funcional (.ipynb) con base de conocimiento, agente y celda de chat con 5 ejemplos
-  README con descripción del dominio, instrucciones de ejecución y justificación del system prompt
-  Presentación oral de 5-10 min con demo en vivo el día de entrega

 **BONUS: URL pública de la app en Streamlit Cloud**



¿Preguntas?

Tiempo libre para empezar el proyecto

Empieza eligiendo tu dominio y recopilando los documentos — es el paso más crítico

Consigue tu API key de Gemini en aistudio.google.com antes de salir de clase

El documento del proyecto está disponible para descargar